# CATEGORY 11: CI/CD & Jenkins Troubleshooting Scenarios for Interviews

> ⚠ **Interview Reality Check**: Troubleshooting questions reveal whether you have *real production experience* or just theoretical knowledge. Interviewers look for structured thinking (observe → hypothesize → isolate → fix → prevent), not just "I googled the error."

---

## How to Answer Troubleshooting Questions in Interviews

Use this framework every time:

```
1. OBSERVE    → What are the symptoms? Logs, metrics, error messages
2. HYPOTHESIZE → What are the possible root causes?
3. ISOLATE    → How do you narrow it down?
4. FIX        → What's the immediate fix?
5. PREVENT    → How do you stop it happening again?
```

---

## SCENARIO 11.1: Build Passes Locally but Fails in Jenkins

**⚠ Extremely common — asked in almost every interview**

### Symptoms

- Developer runs tests locally: all green
- Same code in Jenkins: fails with compilation errors, test failures, or missing dependencies

### Root Causes & Diagnosis

```
Possible Causes:
├── Environment differences (JDK version, Node version, OS)
├── Missing environment variables or secrets
├── Hardcoded absolute paths (e.g., /Users/dev/project)
├── Local uncommitted changes not pushed
├── Different dependency versions (no lockfile, or lockfile ignored)
├── Time-sensitive tests relying on local timezone
├── Tests relying on local file system state
└── Docker image version drift

Stale local cache: Developer has old artifacts cached in ~/.m2 or ~/.npm
that no longer exist in the remote registry. Jenkins starts fresh and
fails to resolve them. Fix: proxy all external dependencies through
internal Nexus/Artifactory.
```

### Diagnosis Steps

```
# Step 1: Check Jenkins agent Java/Node/Python version
java -version
node --version
python3 --version

# Step 2: Compare environment variables
# In Jenkinsfile, add a debug stage:
stage('Debug Env') {
    steps {
        sh 'env | sort'
        sh 'java -version'
        sh 'mvn --version'
    }
}

# Step 3: Check if tests use hardcoded paths
grep -r "/Users/" src/test/
grep -r "/home/" src/test/

# Step 4: Validate dependency lockfile is committed
git status package-lock.json   # Node
git status Pipfile.lock        # Python
git status pom.xml             # Maven
```

### Fix

```
// Declarative Pipeline — enforce consistent environment
pipeline {
    agent {
        docker {
            image 'maven:3.8.6-openjdk-17'  // Pin exact version
            args '-v $HOME/.m2:/root/.m2'    // Cache Maven deps
        }
    }
    environment {
        JAVA_HOME = tool 'JDK17'            // Use Jenkins-managed tool
        MAVEN_OPTS = '-Xmx1024m'
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean verify'
            }
        }
    }
}
```

### Interview Answer (Crisp)

> "When a build passes locally but fails in Jenkins, the root cause is almost always an environment mismatch — different JDK versions, missing environment variables, or dependency drift. I start by printing the full environment in Jenkins, comparing tool versions, and checking if tests have hardcoded local paths. The long-term fix is containerizing the build environment with pinned image versions so the build runs identically everywhere."

> "Prevention: Always route dependency resolution through an internal Nexus/Artifactory proxy. This ensures all artifacts are cached internally, making builds reproducible regardless of external registry availability or local cache state."

---

## SCENARIO 11.2: Jenkins Pipeline Stuck / Hanging Indefinitely

### Symptoms

- Pipeline shows "running" for hours with no output
- Build never fails, never completes
- Console log stops at a specific point

### Root Causes

```
Possible Causes:
├── sh/bat step waiting for interactive input (password prompt, Y/N)
├── Test waiting for network resource that never responds
├── SSH connection hanging (no timeout set)
├── mvn/npm waiting for user confirmation
├── Gradle daemon lock issue
└── Docker build waiting on stdin
```

### Diagnosis Steps

```
# On the Jenkins agent machine, find the hanging process
ps aux | grep -E "mvn|npm|gradle|docker"

# Check if it's waiting for stdin
lsof -p <PID> | grep -E "stdin|pipe"

# Check network connectivity from agent
curl -v --max-time 10 https://your-dependency-server.com
```

### Fix

```
// Declarative Pipeline — always set timeouts
pipeline {
```

```
    options {
        timeout(time: 30, unit: 'MINUTES')  // Global timeout
    }
    stages {
        stage('Build') {
            options {
                timeout(time: 10, unit: 'MINUTES')  // Stage-level timeout
            }
            steps {
                // Pass -y or --batch-mode to suppress interactive prompts
                sh 'mvn --batch-mode -DskipTests clean package'
                sh 'npm ci --no-interactive'
                sh 'apt-get install -y some-package'  // -y flag is
critical
            }
        }
    }
}

// Scripted Pipeline
node {
    timeout(time: 10, unit: 'MINUTES') {
        stage('Build') {
            sh 'mvn --batch-mode clean package'
        }
    }
}
```

### Gotchas ⚠

- `timeout()` in Jenkins kills the build but may leave zombie processes on the agent — always combine with post-cleanup steps
- Some Docker builds inherit stdin from the parent process — always pass `--no-stdin` or equivalent
- Gradle can deadlock if two builds share the same workspace — use `--no-daemon` in CI

---

## SCENARIO 11.3: Out of Memory (OOM) / Jenkins Controller Crash

### Symptoms

- Jenkins becomes unresponsive
- `java.lang.OutOfMemoryError: Java heap space` in logs
- New builds won't start, existing builds frozen
- `jenkins.log` shows GC thrashing

### Root Causes

```
Possible Causes:
├── Too many concurrent builds on controller (builds should run on agents)
├── Build logs stored in memory (large console output)
```

```
├── Plugin memory leaks
├── Insufficient heap allocation (-Xmx too low)
├── Old builds/artifacts not cleaned up (JENKINS_HOME fills disk)
└── Pipeline Groovy scripts loading too much data into variables
```

### Diagnosis Steps

```
# Check current JVM heap usage
curl -s http://localhost:8080/monitoring?key=YOURKEY | grep heap

# Check JENKINS_HOME disk usage
du -sh /var/jenkins_home/jobs/*/builds/
du -sh /var/jenkins_home/

# Check number of loaded builds in memory
# Jenkins UI: Manage Jenkins → System Information → Memory

# Get GC logs (add to JVM startup)
-
Xlog:gc*:file=/var/log/jenkins/gc.log:time,level,tags:filecount=5,filesize
=20m
```

### Fix

```
# 1. Increase heap in /etc/default/jenkins (or JAVA_OPTS)
JAVA_OPTS="-Xms2g -Xmx4g -XX:+UseG1GC -XX:MaxGCPauseMillis=200"

# 2. Set build retention policy in Jenkinsfile
pipeline {
    options {
        buildDiscarder(logRotator(
            numToKeepStr: '10',        // Keep last 10 builds
            artifactNumToKeepStr: '5', // Keep artifacts for last 5
            daysToKeepStr: '30'        // Or by age
        ))
    }
    ...
}

# 3. Move all builds off controller — agents only
# In Jenkins UI: Manage Jenkins → Configure System
# → Set "# of executors" on controller to 0
```

### Interview Answer

> "Jenkins OOM is usually caused by running builds directly on the controller instead of agents, or by not setting log retention policies. First I'd increase the heap with `-Xmx`, set executors on the

> controller to zero, and add `buildDiscarder` to all pipelines. For production, I'd also expose JVM metrics to Prometheus and alert before it OOMs rather than after."

---

## SCENARIO 11.4: Flaky Tests — Tests That Randomly Pass/Fail

### Symptoms

- Same commit: sometimes green, sometimes red
- No code changes between runs with different results
- Tests fail in CI but pass locally

### Root Causes

```
Possible Causes:
├── Race conditions in async code
├── Tests sharing mutable state (DB, file system, global vars)
├── Time-dependent tests (e.g., "is it business hours?")
├── Test ordering dependency (test B assumes test A ran first)
├── External service dependency (network calls to real APIs)
├── Port conflicts from parallel test runs
└── Timezone differences between dev machine and CI agent
```

### Fix

```
// Retry flaky tests automatically (short-term bandage, not a cure)
pipeline {
    stages {
        stage('Test') {
            steps {
                retry(3) {  // Retry up to 3 times
                    sh 'mvn test'
                }
            }
        }
    }
}

// Better fix: quarantine known flaky tests
// In Maven, use @Ignore or test groups
// Report flaky test metrics over time
post {
    always {
        junit testResults: '**/target/surefire-reports/*.xml',
              allowEmptyResults: true,
              skipPublishingChecks: false
    }
}
```

### Diagnosis Script

```
# Run the test N times to measure flakiness rate
for i in $(seq 1 10); do
    mvn test -pl :module-name -Dtest=FlakyTestClass 2>&1 | \
    tail -1 >> flakiness_results.txt
done
grep -c "BUILD SUCCESS" flakiness_results.txt
```

### Gotcha ⚠

Using `retry()` hides flakiness — it makes the build green but the underlying problem grows. Always track flaky test rate as a metric and treat it as a P2 bug.

---

## SCENARIO 11.5: Credentials/Secrets Showing in Build Logs

**⚠ Security-critical — always comes up in senior interviews**

### Symptoms

- API keys, passwords visible in console output
- Credentials appear in error messages
- Secret accidentally echoed by a script

### What NOT to Do

```
// ❌ NEVER - hardcode secrets
environment {
    API_KEY = "sk-prod-abc123xyz"
}

// ❌ NEVER - echo credentials
sh 'echo $DB_PASSWORD'

// ❌ NEVER - print entire environment
sh 'env'  // This dumps ALL env vars including secrets
```

### Correct Approach

```
// ✅ Use credentials binding - Jenkins masks the value in logs
pipeline {
    environment {
        // Credentials are masked in console output automatically
        AWS_CREDS = credentials('aws-production-creds')  //
Username/password type
        DB_PASSWORD = credentials('db-prod-password')    // Secret text
```

```
type
    }
    stages {
        stage('Deploy') {
            steps {
                withCredentials([
                    string(credentialsId: 'sonar-token', variable:
'SONAR_TOKEN'),
                    usernamePassword(
                        credentialsId: 'nexus-creds',
                        usernameVariable: 'NEXUS_USER',
                        passwordVariable: 'NEXUS_PASS'
                    )
                ]) {
                    sh '''
                        # Jenkins replaces $SONAR_TOKEN with **** in logs
                        mvn sonar:sonar -Dsonar.login=$SONAR_TOKEN
                    '''
                }
            }
        }
    }
}
```

### If a Secret Is Already Leaked

```
# Immediate steps:
# 1. Rotate the credential immediately
# 2. Mask the Jenkins build log retrospectively (if possible)
# 3. Audit access logs for the exposed credential
# 4. If in Git history, rotate AND rewrite history (git filter-branch or
BFG Repo Cleaner)
bfg --replace-text passwords.txt  # BFG Repo Cleaner
git push --force --all
```

---

## SCENARIO 11.6: Pipeline Fails Due to Workspace Pollution

### Symptoms

- Build fails with "file already exists" error
- Stale artifacts from previous builds cause test failures
- `git checkout` fails due to dirty workspace

### Root Causes

```
Possible Causes:
├── Previous build was interrupted, leaving temp files
```

```
├── Concurrent builds sharing a workspace
├── Agent reused without cleaning workspace
└── Generated files committed accidentally
```

Fix

```
// Option 1: Clean before build
pipeline {
    options {
        skipDefaultCheckout(true)  // Don't auto-checkout
    }
    stages {
        stage('Checkout') {
            steps {
                cleanWs()          // Clean workspace before checkout
                checkout scm       // Then checkout
            }
        }
    }
    post {
        always {
            cleanWs(cleanWhenNotBuilt: false,
                    deleteDirs: true,
                    disableDeferredWipeout: true)
        }
    }
}

// Option 2: Use git clean
steps {
    sh 'git clean -fdx'   // Remove all untracked files
    sh 'git reset --hard' // Reset any modifications
}

// Option 3: Custom workspace per build
node {
    ws("workspace/${env.JOB_NAME}/${env.BUILD_NUMBER}") {
        // Each build gets its own workspace
        checkout scm
        sh 'mvn clean package'
    }
}
```

---

## SCENARIO 11.7: Jenkins Agent Goes Offline / Disconnects During Build

### Symptoms

- Build fails with "Agent went offline"
- "Connection was broken" error mid-pipeline

- Agent shows as "Temporarily offline" in Jenkins UI

### Root Causes

```
Possible Causes:
├── Agent ran out of disk space
├── Agent ran out of memory (OOM killed)
├── Network interruption between controller and agent
├── SSH timeout (too long idle during test)
├── Agent JVM crash
├── EC2/cloud agent preempted or spot instance reclaimed
└── Docker container exited unexpectedly
```

### Diagnosis Steps

```bash
# On the agent machine:
# 1. Check disk space
df -h

# 2. Check if OOM killed
dmesg | grep -i "oom\|killed"
journalctl -u jenkins-agent | grep -i "error\|killed"

# 3. Check agent log
cat /var/log/jenkins/agent.log | tail -100

# 4. Check network
ping <jenkins-controller-ip>
telnet <jenkins-controller-ip> 50000  # JNLP port
```

### Fix

```groovy
// Add retry logic for agent reconnection
pipeline {
    agent {
        label 'linux-agent'
    }
    options {
        // Retry the entire pipeline on agent failure
        retry(2)
    }
    stages {
        stage('Build') {
            steps {
                sh 'df -h'  // Log disk before build
                sh 'free -m' // Log memory before build
            }
        }
```

```
    }
}
```

```
# Prevent disk OOM on agent — add to crontab
0 2 * * * find /var/jenkins/workspace -mtime +7 -exec rm -rf {} +
0 2 * * * docker system prune -f
```

For Kubernetes Agents (Common in Senior Interviews)

```
pipeline {
    agent {
        kubernetes {
            yaml '''
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: maven
    image: maven:3.8.6-openjdk-17
    resources:
      requests:
        memory: "1Gi"
        cpu: "500m"
      limits:
        memory: "2Gi"     # OOM kill threshold
        cpu: "1000m"
    volumeMounts:
    - name: maven-cache
      mountPath: /root/.m2
  volumes:
  - name: maven-cache
    emptyDir: {}
'''
        }
    }
}
```

---

## SCENARIO 11.8: Docker Build Fails in Jenkins Pipeline

### Symptoms

- `docker: command not found` on agent
- `permission denied while trying to connect to the Docker daemon`
- `Cannot connect to the Docker daemon at unix:///var/run/docker.sock`

### Root Causes

```
Possible Causes:
├── Docker not installed on agent
├── Jenkins user not in docker group
├── Socket permission issue
├── Docker-in-Docker (DinD) not configured
└── Wrong container runtime (containerd vs Docker)
```

Fix

```
# Add jenkins user to docker group (requires restart)
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins

# Verify
sudo -u jenkins docker ps
```

```
// Approach 1: Mount Docker socket (simplest, not most secure)
pipeline {
    agent {
        docker {
            image 'docker:24.0'
            args '-v /var/run/docker.sock:/var/run/docker.sock'
        }
    }
    stages {
        stage('Build Image') {
            steps {
                sh 'docker build -t myapp:${BUILD_NUMBER} .'
            }
        }
    }
}

// Approach 2: Docker-in-Docker with Kubernetes
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  containers:
  - name: docker
    image: docker:24.0-dind
    securityContext:
      privileged: true
    env:
    - name: DOCKER_TLS_CERTDIR
      value: ""
'''
```

```
        }
    }
}

// Approach 3: Kaniko (no Docker socket needed — preferred in k8s)
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  containers:
  - name: kaniko
    image: gcr.io/kaniko-project/executor:latest
    command: ["cat"]
    tty: true
'''
        }
    }
    stages {
        stage('Build') {
            steps {
                container('kaniko') {
                    sh '/kaniko/executor --context . --destination
myrepo/myapp:${BUILD_NUMBER}'
                }
            }
        }
    }
}
```

### Gotcha ⚠

Mounting `/var/run/docker.sock` means the container has root access to the host's Docker daemon. In production, prefer Kaniko, Buildah, or img for rootless builds.

---

## SCENARIO 11.9: Git Checkout Fails / Authentication Issues

### Symptoms

- `fatal: could not read Username for 'https://github.com'`
- `Permission denied (publickey)`
- `remote: Repository not found`

### Root Causes

```
Possible Causes:
├── SSH key not added to Jenkins credentials
├── Known hosts file missing (SSH strict host checking)
├── Personal Access Token expired
├── Wrong credential ID referenced in Jenkinsfile
```

```
├── Proxy blocking outbound connections
└── Agent doesn't have git installed
```

Fix

```
// Correct way to reference credentials in Jenkinsfile
pipeline {
    stages {
        stage('Checkout') {
            steps {
                // Method 1: Using SCM configured in job
                checkout scm

                // Method 2: Explicit with credentials
                checkout([
                    $class: 'GitSCM',
                    branches: [[name: '*/main']],
                    userRemoteConfigs: [[
                        url: 'https://github.com/org/repo.git',
                        credentialsId: 'github-pat-token'  // Must match
credential ID in Jenkins
                    ]]
                ])

                // Method 3: SSH
                checkout([
                    $class: 'GitSCM',
                    branches: [[name: '*/main']],
                    userRemoteConfigs: [[
                        url: 'git@github.com:org/repo.git',
                        credentialsId: 'jenkins-ssh-key'
                    ]]
                ])
            }
        }
    }
}
```

```
# Fix SSH known hosts issue
# Run on agent as jenkins user:
ssh-keyscan github.com >> ~/.ssh/known_hosts
ssh-keyscan bitbucket.org >> ~/.ssh/known_hosts

# Or disable strict host checking (not recommended for production)
# In Jenkins: Manage Jenkins → Configure Global Security
# → Git Host Key Verification: Accept first connection
```

---

## SCENARIO 11.10: Pipeline Shared Library Not Loading / Import Errors

**⚠ Frequently asked for senior Jenkins roles**

### Symptoms

- `groovy.lang.MissingPropertyException: No such property: myLib`
- `WorkflowScript: Unable to resolve class`
- `ERROR: Could not find or load @Library`

### Root Causes

```
Possible Causes:
├── Library not configured in Jenkins Global Configuration
├── Wrong library name in @Library annotation
├── Branch/version mismatch
├── vars/ directory has wrong file naming (must be .groovy, lowercase)
├── Class in src/ missing package declaration
└── Sandbox restrictions blocking library calls
```

### Correct Library Structure

```
(shared-library-repo)/
├── vars/
│   ├── deployApp.groovy      # Called as deployApp(...) in pipeline
│   ├── buildDocker.groovy
│   └── notifySlack.groovy
├── src/
│   └── com/
│       └── myorg/
│           └── Utils.groovy  # import com.myorg.Utils
└── resources/
    └── templates/
        └── deployment.yaml
```

```
// vars/deployApp.groovy
def call(Map config) {
    pipeline {
        agent { label config.agent ?: 'linux' }
        stages {
            stage('Deploy') {
                steps {
                    sh "kubectl apply -f ${config.manifestPath}"
                }
            }
        }
    }
```

```
}

// Using in Jenkinsfile
@Library('my-shared-library@main') _   // @main = branch/tag

deployApp(
    agent: 'kubernetes',
    manifestPath: 'k8s/production/'
)
```

### Diagnosis

```
// Test if library loads at all
@Library('my-shared-library') _

pipeline {
    agent any
    stages {
        stage('Test') {
            steps {
                // If this runs, library loaded
                echo "Library loaded successfully"
                // Try calling a var
                script {
                    echo myVar.someFunction()
                }
            }
        }
    }
}
```

---

## SCENARIO 11.11: SonarQube Quality Gate Blocking Pipeline

### Symptoms

- Pipeline fails at SonarQube stage with "Quality Gate FAILED"
- Pipeline hangs waiting for Quality Gate result
- SonarQube analysis completes but Jenkins doesn't receive result

### Root Causes

```
Possible Causes:
├── New code coverage below threshold
├── Critical/blocker issues introduced
├── Webhook from SonarQube not configured (pipeline waits forever)
├── SonarQube token expired
```

```
└── Branch not analyzed in SonarQube (Community Edition branch
restrictions)
```

Fix

```
pipeline {
    stages {
        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube-Production') {
                    sh 'mvn sonar:sonar'
                }
            }
        }
        stage('Quality Gate') {
            steps {
                // Wait up to 5 minutes for webhook callback from
SonarQube
                timeout(time: 5, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }
    }
}
```

```
# Configure SonarQube webhook (must be done once in SonarQube):
# SonarQube UI → Administration → Webhooks → Create
# URL: http://<jenkins-url>/sonarqube-webhook/
# Secret: (optional, for security)
```

Handling Quality Gate Failures Gracefully

```
stage('Quality Gate') {
    steps {
        script {
            def qg = waitForQualityGate()
            if (qg.status != 'OK') {
                if (env.BRANCH_NAME == 'main') {
                    error "Quality Gate failed on main: ${qg.status}"
                } else {
                    // Warn but don't fail on feature branches
                    unstable "Quality Gate failed: ${qg.status}"
                }
            }
        }
```

```
    }
}
```

---

## SCENARIO 11.12: Artifact Upload to Nexus/Artifactory Fails

### Symptoms

- `Could not transfer artifact` error
- 401 Unauthorized when pushing artifacts
- Artifact uploads but wrong version or to wrong repository

### Root Causes

```
Possible Causes:
├── Credentials not configured or expired
├── Repository policy: releases repo rejects SNAPSHOT versions
├── Nexus storage quota exceeded
├── Wrong repository URL (releases vs snapshots)
└── pom.xml distributionManagement not set
```

### Fix

```
// Maven settings with credentials
pipeline {
    stages {
        stage('Publish') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId: 'nexus-credentials',
                        usernameVariable: 'NEXUS_USER',
                        passwordVariable: 'NEXUS_PASS'
                    )
                ]) {
                    sh '''
                        mvn deploy \
                          -DskipTests \
                          -Dnexus.username=$NEXUS_USER \
                          -Dnexus.password=$NEXUS_PASS \
                          --settings settings.xml
                    '''
                }
            }
        }
    }
}
```

```
<!-- settings.xml with server config -->
<settings>
  <servers>
    <server>
      <id>nexus-releases</id>
      <username>${nexus.username}</username>
      <password>${nexus.password}</password>
    </server>
  </servers>
</settings>
```

---

## SCENARIO 11.13: Multi-Branch Pipeline Not Detecting New Branches

### Symptoms

- New feature branch pushed to Git — no pipeline created in Jenkins
- Deleted branches still show in Jenkins
- PR builds not triggering

### Root Causes

```
Possible Causes:
├── Scan interval too long (or disabled)
├── Webhook not configured in GitHub/GitLab
├── Jenkinsfile not present on new branch
├── Branch filter regex too restrictive
└── GitHub/GitLab plugin not installed or misconfigured
```

### Fix

```
// In Jenkins job configuration (or via JCasC):
// Multibranch Pipeline → Scan Multibranch Pipeline Triggers
// Set: Periodically if not otherwise run → 1 hour

// Better: Configure GitHub webhook
// GitHub Repo → Settings → Webhooks → Add webhook
// Payload URL: http://<jenkins>/github-webhook/
// Content type: application/json
// Events: Push, Pull Request

// Branch filtering in Multibranch Pipeline
// Behaviors → Filter by name (with regular expression):
// Include: main|develop|release/.*|feature/.*|PR-\d+
```

```
// Jenkinsfile on the branch — required for discovery
// Even a minimal one works:
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo "Building branch: ${env.BRANCH_NAME}"
            }
        }
    }
}
```

---

## SCENARIO 11.14: Parallel Stages Failing with Confusing Error Messages

**⚠ Common trick question**

### Symptoms

- One parallel branch fails but another continues — final error message is confusing
- `failFast: true` kills all branches when one fails
- Parallel stages compete for the same resource

### Demonstration

```
pipeline {
    stages {
        stage('Parallel Tests') {
            parallel {
                stage('Unit Tests') {
                    steps {
                        sh 'mvn test -Dtest=UnitTestSuite'
                    }
                }
                stage('Integration Tests') {
                    steps {
                        sh 'mvn test -Dtest=IntegrationTestSuite'
                    }
                }
                stage('Security Scan') {
                    steps {
                        sh './run-security-scan.sh'
                    }
                }
            }
            // failFast: true — if ANY branch fails, abort ALL
            // failFast: false (default) — all branches run to completion
        }
```

```
    }
}
```

### Common Gotcha ⚠

```
// PROBLEM: Parallel stages on same agent writing to same workspace
parallel {
    stage('Test A') {
        steps {
            sh 'echo result > output.txt'  // Both write to same file!
        }
    }
    stage('Test B') {
        steps {
            sh 'echo result > output.txt'  // Race condition!
        }
    }
}

// FIX: Use separate agents or separate directories
parallel {
    stage('Test A') {
        agent { label 'agent-1' }
        steps {
            sh 'echo result > output-a.txt'
        }
    }
    stage('Test B') {
        agent { label 'agent-2' }
        steps {
            sh 'echo result > output-b.txt'
        }
    }
}
```

---

## SCENARIO 11.15: Stash/Unstash Failing Across Agents

**⚠ Commonly misunderstood**

### Symptoms

- `unstash` finds no stash
- Stashed files corrupt or incomplete
- Large stashes causing Jenkins controller memory issues

### How Stash Works

```
stash → Files sent to Jenkins Controller memory/disk
unstash → Files retrieved FROM Controller to current agent workspace

Problem: Controller becomes bottleneck for large stashes
```

Fix

```
pipeline {
    agent none  // Agents assigned per stage
    stages {
        stage('Build') {
            agent { label 'build-agent' }
            steps {
                sh 'mvn clean package -DskipTests'
                stash name: 'build-artifacts',
                      includes: 'target/*.jar',
                      allowEmpty: false
            }
        }
        stage('Test') {
            agent { label 'test-agent' }  // Different agent
            steps {
                unstash 'build-artifacts'   // Retrieve from controller
                sh 'mvn test'
            }
        }
    }
}

// Alternative for large artifacts: use external storage
// Upload to S3/Nexus instead of stash
stage('Build') {
    steps {
        sh 'mvn clean package'
        sh 'aws s3 cp target/app.jar s3://my-
bucket/builds/${BUILD_NUMBER}/app.jar'
    }
}
stage('Test') {
    steps {
        sh 'aws s3 cp s3://my-bucket/builds/${BUILD_NUMBER}/app.jar
./app.jar'
        sh 'java -jar app.jar --test'
    }
}
```

---

## SCENARIO 11.16: Blue/Green Deployment Rollback Triggered

### Symptoms

- New deployment (green) is live but showing errors
- Need to switch traffic back to old deployment (blue)
- Database migration has already run

### Rollback Strategy

```
pipeline {
    environment {
        CURRENT_ENV = sh(script: 'cat /etc/current-env', returnStdout:
true).trim()
        NEW_ENV = "${CURRENT_ENV == 'blue' ? 'green' : 'blue'}"
    }
    stages {
        stage('Deploy Green') {
            steps {
                sh "kubectl set image deployment/app-${NEW_ENV}
app=myapp:${BUILD_NUMBER}"
                sh "kubectl rollout status deployment/app-${NEW_ENV} --
timeout=5m"
            }
        }
        stage('Health Check') {
            steps {
                script {
                    def healthy = sh(
                        script: "curl -sf http://app-${NEW_ENV}-
internal/health",
                        returnStatus: true
                    ) == 0
                    if (!healthy) {
                        error "Health check failed — aborting before
traffic switch"
                    }
                }
            }
        }
        stage('Switch Traffic') {
            steps {
                sh "kubectl patch service app --patch '{\"spec\":
{\"selector\":{\"env\":\"${NEW_ENV}\"}}}'"
                sh "echo ${NEW_ENV} > /etc/current-env"
            }
        }
        stage('Monitor') {
            steps {
                timeout(time: 5, unit: 'MINUTES') {
                    sh './scripts/check-error-rate.sh'  // Monitor for 5
min post-deploy
                }
            }
```

```
        }
    }
    post {
        failure {
            // Auto-rollback: switch traffic back to old env
            sh "kubectl patch service app --patch '{\"spec\":
{\"selector\":{\"env\":\"${CURRENT_ENV}\"}}}'"
            slackSend channel: '#alerts',
                      color: 'danger',
                      message: "Deployment FAILED — rolled back to
${CURRENT_ENV}"
        }
    }
}
```

⚠ WARNING — kubectl rollout undo + ArgoCD selfHeal conflict: If ArgoCD has selfHeal: true enabled, it will revert your kubectl rollout undo back to the broken version within 3 minutes because Git still points to the broken image tag. Always revert Git first when using ArgoCD — never just the cluster directly.

### Database Migration Gotcha ⚠

> "If a DB migration already ran and is not backward-compatible, you cannot simply route traffic back. You must write expand-contract migrations: expand the schema (add new column), deploy, then contract (remove old column in a later release). This is why backward-compatible schema changes are a hard requirement for blue/green deployments."

---

## SCENARIO 11.17: Jenkins Job DSL / JCasC Configuration Not Applying

### Symptoms

- Changes to JCasC YAML not reflected in Jenkins
- Job DSL scripts not creating/updating jobs
- "Configuration unchanged" despite YAML modifications

### Fix

```
# Force JCasC reload
# Jenkins UI: Manage Jenkins → Configuration as Code → Reload Existing
Configuration

# Or via API
curl -X POST http://localhost:8080/configuration-as-code/reload \
  --user admin:$JENKINS_API_TOKEN

# Validate YAML before applying
curl -X POST http://localhost:8080/configuration-as-code/check \
  --user admin:$JENKINS_API_TOKEN \
  -H "Content-Type: application/json" \
  -d @casc.yaml
```

```
# Example JCasC — common mistake: wrong indentation
# WRONG:
jenkins:
  systemMessage: "Hello"
    numExecutors: 0    # ← Wrong indentation causes silent ignore

# CORRECT:
jenkins:
  systemMessage: "Hello"
  numExecutors: 0
```

---

## SCENARIO 11.18: Kubernetes Pod Agent Not Starting

**⚠ Asked frequently for cloud-native Jenkins roles**

### Symptoms

- Pipeline says "waiting for agent to connect"
- Pod shown as Pending or CrashLoopBackOff in Kubernetes
- `kubectl get pods -n jenkins` shows ERROR state

### Diagnosis

```
# Check pod status
kubectl get pods -n jenkins -l jenkins=agent

# Get detailed pod info
kubectl describe pod <pod-name> -n jenkins

# Common issues shown in describe output:
# - Insufficient CPU/memory (node pressure)
# - Image pull error (wrong image name, no pull secret)
# - Volume mount failure
# - Security context conflict

# Check Jenkins Kubernetes plugin logs
kubectl logs <jenkins-controller-pod> -n jenkins | grep -i
"kubernetes\|pod\|agent"
```

### Fix

```
pipeline {
    agent {
        kubernetes {
            inheritFrom 'default'
```

```
            yaml """
apiVersion: v1
kind: Pod
metadata:
  labels:
    jenkins: agent
spec:
  serviceAccountName: jenkins-agent  # Needs correct RBAC
  imagePullSecrets:
  - name: docker-registry-secret     # For private registries
  containers:
  - name: jnlp                       # Must be named 'jnlp'
    image: jenkins/inbound-agent:latest
    resources:
      requests:
        cpu: 100m
        memory: 256Mi
      limits:
        cpu: 500m
        memory: 512Mi
  - name: maven
    image: maven:3.8.6-openjdk-17
    command: ['cat']
    tty: true
    resources:
      requests:
        cpu: 200m
        memory: 512Mi
"""
        }
    }
    stages {
        stage('Build') {
            steps {
                container('maven') {
                    sh 'mvn clean package'
                }
            }
        }
    }
}
```

---

## SCENARIO 11.19: Pipeline Script Approval Blocking Execution

### Symptoms

- Pipeline fails with `Scripts not permitted to use method...`
- Groovy method call rejected by sandbox
- Admin must approve scripts constantly

### Root Cause

Jenkins Groovy sandbox restricts which methods scripts can call. Unapproved methods require admin approval in Script Approval.

### Fix

```
# Immediate fix: Approve in Jenkins UI
# Manage Jenkins → In-process Script Approval → Approve pending signatures

# Long-term: Use Pipeline Steps instead of raw Groovy
```

```
// ❌ This may require script approval
def result = ["ls", "-la"].execute().text

// ✅ Use sh step instead (approved by default)
def result = sh(script: 'ls -la', returnStdout: true).trim()

// ❌ This requires approval
import groovy.json.JsonSlurper
def data = new JsonSlurper().parseText(json)

// ✅ Use readJSON (Pipeline Utility Steps plugin)
def data = readJSON text: json
```

### Gotcha ⚠

Using `@NonCPS` annotation bypasses some sandbox restrictions, but the method cannot use Pipeline steps. Use it only for pure Groovy data manipulation functions.

```
@NonCPS
def parseCSV(String content) {
    // Pure Groovy - no Pipeline steps here
    return content.split('\n').collect { it.split(',') }
}
```

---

## SCENARIO 11.20: Deployment to Production Without Proper Approval

### Scenario

"Your pipeline accidentally deployed to production without a manual approval gate. How do you fix this?"

### Root Cause Analysis

```
// Bug: approval only on first deploy, not subsequent ones
pipeline {
```

```
    stages {
        stage('Approve') {
            when {
                branch 'main'
                expression { return currentBuild.number == 1 }  // ← BUG:
only build #1!
            }
            steps {
                input message: 'Deploy to production?'
            }
        }
        stage('Deploy') {
            steps {
                sh './deploy.sh production'
            }
        }
    }
}
```

Correct Implementation

```
pipeline {
    stages {
        stage('Approve Production Deployment') {
            when {
                branch 'main'
                // No expression condition — ALWAYS require approval on
main
            }
            steps {
                timeout(time: 24, unit: 'HOURS') {  // Don't block forever
                    input(
                        message: "Deploy ${env.BUILD_NUMBER} to
PRODUCTION?",
                        ok: 'Deploy',
                        submitter: 'release-team,sre-oncall',  // Restrict
who can approve
                        parameters: [
                            choice(
                                choices: ['Deploy', 'Cancel'],
                                description: 'Action',
                                name: 'APPROVAL_ACTION'
                            )
                        ]
                    )
                }
            }
        }
        stage('Deploy Production') {
            when {
                branch 'main'
```

```
            }
            steps {
                sh './deploy.sh production'
            }
        }
    }
}
```

---

## SCENARIO 11.21: High Build Queue — Builds Waiting for Hours

### Symptoms

- Builds queued for 2+ hours
- "waiting for next available executor" message
- Agents available but not picking up jobs

### Diagnosis

```
Root Causes:
├── All agents are busy (scaling issue)
├── Agent label mismatch — job requires label no agent has
├── Agent is online but marked as suspended
├── Build queue processing is stuck (Jenkins bug)
└── Too many concurrent builds from one triggering source
```

### Fix

```
// Check label usage — common mistake
pipeline {
    agent { label 'linux-amd64' }  // Label must EXACTLY match agent label
    // If agent has label 'linux' but not 'linux-amd64' — will queue
forever
}

// Fix: use more flexible label expressions
pipeline {
    agent { label 'linux && docker' }  // Agent needs BOTH labels
    // or
    agent { label 'linux || mac' }     // Agent needs EITHER label
}
```

```
# Jenkins CLI: list agents and their labels
java -jar jenkins-cli.jar -s http://localhost:8080 \
  -auth admin:$TOKEN \
  list-nodes
```

```
# Check queue via API
curl -s http://localhost:8080/queue/api/json \
  --user admin:$TOKEN | jq '.items[].why'
```

### Scaling Fix

```
// JCasC — configure EC2 auto-scaling agents
jenkins:
  clouds:
  - amazonEC2:
      name: "ec2-agents"
      region: "us-east-1"
      templates:
      - ami: "ami-0abcdef1234567890"
        instanceType: T3Medium
        labelString: "linux docker"
        minimumNumberOfInstances: 0
        maximumTotalUses: 1  # Ephemeral — terminate after 1 build
```

---

## SCENARIO 11.22: Canary Deployment Showing Errors — How to Respond

### Scenario

"You deployed a canary to 10% of traffic. Error rate jumped from 0.1% to 2%. Walk me through your response."

### Step-by-Step Response

```
Phase 1 — Detect (0-2 minutes)
├── PagerDuty alert fires: error rate > 1%
├── Open Grafana: compare canary pods vs stable pods error rates
└── Confirm: canary pods = 2% errors, stable = 0.1% — canary is the issue

Phase 2 — Isolate (2-5 minutes)
├── Check canary pod logs: kubectl logs -l version=canary --tail=100
├── Look for new exception class or stack trace not in stable
└── Check if error is for all users or specific region/user segment

Phase 3 — Rollback (5-10 minutes)
├── Route 0% traffic to canary immediately
├── Keep canary pods running (for forensics — don't delete yet)
└── Confirm error rate returns to baseline

Phase 4 — Root Cause (post-incident)
├── Compare canary code diff
├── Reproduce in staging
└── Write post-mortem
```

```
// Automated canary analysis in pipeline
stage('Canary Analysis') {
    steps {
        script {
            // Deploy canary to 10%
            sh 'kubectl set image deployment/app-canary
app=myapp:${BUILD_NUMBER}'
            sh 'kubectl patch virtualservice app --patch @canary-10-
percent.yaml'

            // Wait and measure
            sleep(time: 5, unit: 'MINUTES')

            // Query error rate from Prometheus
            def errorRate = sh(
                script: '''
                    curl -s "http://prometheus:9090/api/v1/query?query=
                    rate(http_errors_total{version=\"canary\"}[5m])
                    /rate(http_requests_total{version=\"canary\"}[5m])" \
                    | jq '.data.result[0].value[1]' -r
                ''',
                returnStdout: true
            ).trim().toFloat()

            if (errorRate > 0.01) {  // > 1% error rate
                sh 'kubectl patch virtualservice app --patch @canary-0-
percent.yaml'
                error "Canary aborted: error rate ${errorRate * 100}%
exceeds threshold"
            }
        }
    }
}
```

---

## SCENARIO 11.23: Pipeline Fails on Database Migration

**⚠ Database migrations in CI/CD — a favorite senior interview topic**

### Symptoms

- Deployment succeeds but application crashes on startup
- "Table does not exist" or "Column not found" errors
- Rollback fails because schema has changed

### Root Cause Patterns

```
Pattern 1: Non-backward-compatible migration ran BEFORE code deploy
  → New code expects new schema, old code gets new schema → crashes during
deploy

Pattern 2: Migration runs AFTER code deploy
  → Old schema, new code → crashes immediately

Pattern 3: Failed migration leaves schema in partial state
```

Safe Migration Pattern

```
pipeline {
    stages {
        // Step 1: Run EXPAND migration (additive only — new
tables/columns)
        stage('DB Migration - Expand') {
            steps {
                sh 'flyway migrate -
locations=filesystem:migrations/expand'
                sh 'flyway validate'
            }
        }

        // Step 2: Deploy new application code
        stage('Deploy Application') {
            steps {
                sh 'kubectl set image deployment/app
app=myapp:${BUILD_NUMBER}'
                sh 'kubectl rollout status deployment/app --timeout=5m'
            }
        }

        // Step 3: Run CONTRACT migration (cleanup — remove old columns)
        // In a SEPARATE pipeline run, after confirming stability
        stage('DB Migration - Contract') {
            when {
                expression { return params.RUN_CONTRACT_MIGRATION ==
'true' }
            }
            steps {
                sh 'flyway migrate -
locations=filesystem:migrations/contract'
            }
        }
    }
    post {
        failure {
            script {
                // Rollback application — but NOT the migration
(dangerous)
```

```
                    sh 'kubectl rollout undo deployment/app'
                    // Migration rollback requires explicit reverse migration
                    sh 'flyway undo'  // Only if migration is reversible
                }
            }
        }
    }
```

### Interview Answer

> "Database migrations and deployments must be decoupled. The expand-contract pattern ensures backward compatibility: first expand the schema (add new structures without removing old ones), then deploy the application, then contract (remove deprecated structures in a later release). This allows rollback of the application without rolling back the schema, which is often impossible."

---

## SCENARIO 11.24: Jenkins Upgrade Breaks Pipelines

### Symptoms

- After Jenkins upgrade, pipelines fail with new errors
- Plugin compatibility issues
- Groovy syntax that worked before now rejected

### Prevention & Fix

```
# Before upgrade — test in staging Jenkins first
# Check plugin compatibility matrix:
# https://www.jenkins.io/doc/upgrade-guide/

# Take a full backup before upgrade
cp -r /var/jenkins_home /backup/jenkins_home_$(date +%Y%m%d)

# Check which plugins need updating
java -jar jenkins-cli.jar -s http://localhost:8080 \
  -auth admin:$TOKEN \
  list-plugins | grep -v " ok "  # Show only out-of-date

# Update all plugins via CLI
java -jar jenkins-cli.jar -s http://localhost:8080 \
  -auth admin:$TOKEN \
  install-plugin $(java -jar jenkins-cli.jar \
    -s http://localhost:8080 -auth admin:$TOKEN \
    list-plugins | awk '/ / { print $1 }')
```

### Common Upgrade Break Patterns

```
// Groovy sandbox got stricter in newer versions
// Old code (may break):
def data = readFile('file.txt').split('\n')

// New safe version:
def data = readFile(file: 'file.txt').trim().split('\n')

// Pipeline syntax changes — use Pipeline Syntax generator
// Jenkins UI: /pipeline-syntax/ → always generates current valid syntax
```

---

## SCENARIO 11.25: Complete Pipeline Failure Analysis Framework

**This is what separates senior candidates from junior ones**

### The Systematic Approach

```
When a pipeline fails:

1. READ THE ERROR (don't scroll up yet)
   → What is the exact error message?
   → Which stage failed?
   → What was the last successful step?

2. CHECK BUILD HISTORY
   → Did this ever work?
   → What changed between last passing and this failing build?
   → Is this a first-time setup or regression?

3. CORRELATE WITH CHANGES
   → Code changes (git diff)
   → Infrastructure changes (was anything deployed?)
   → Jenkins changes (plugin update, config change)
   → External dependencies (did SonarQube go down? Nexus?)

4. ISOLATE THE FAILURE DOMAIN
   Build failure?    → Code/compiler/dependency issue
   Test failure?     → Code logic or test environment
   Deploy failure?   → Infrastructure/permissions/connectivity
   Post-step failure? → Notification/reporting (usually non-critical)

5. CHECK ENVIRONMENT
   → Agent healthy? (disk, memory)
   → Network reachable? (external services)
   → Credentials valid? (not expired)

6. FIX AND VERIFY
   → Fix root cause (not symptom)
   → Re-run and verify
   → Add monitoring to detect earlier next time
```

---

## SCENARIO 11.26: Build Dependency Failures

**⚠ Extremely common in production — asked frequently in senior interviews**

### Symptoms

- `Could not resolve dependency` or `Package not found`
- Build worked yesterday, fails today with no code changes
- Dependency downloads randomly time out
- `SNAPSHOT` dependency pulls different code each run
- Transitive dependency conflict causing `ClassNotFoundException` at runtime

### Root Causes

```
Possible Causes:
├── External registry down (Maven Central, npm registry, PyPI)
├── Dependency version deleted from registry (left-pad problem)
├── SNAPSHOT dependency resolved to broken nightly build
├── Version range specified instead of pinned version (e.g., ^1.0.0)
├── Transitive dependency version conflict (diamond dependency)
├── Corporate proxy/firewall blocking external registry
├── Artifact cache corrupted on agent
└── Private registry authentication expired
```

---

### Case A: External Registry Down / Dependency Deleted

```
# Symptom
[ERROR] Could not resolve com.example:library:1.4.2
[ERROR] Could not transfer artifact from central
(https://repo.maven.apache.org)

# Diagnosis — test registry reachability from agent
curl -v https://repo.maven.apache.org/maven2/
curl -v https://registry.npmjs.org/

# Check if it's a proxy issue
curl -v --proxy http://corporate-proxy:8080 https://repo.maven.apache.org/
```

**Fix — Route through internal Nexus/Artifactory proxy:**

```
<!-- Maven: settings.xml — always proxy through internal Nexus -->
<settings>
  <mirrors>
    <mirror>
```

```
      <id>nexus-central</id>
      <mirrorOf>central</mirrorOf>
      <url>https://nexus.internal/repository/maven-proxy/</url>
    </mirror>
    <mirror>
      <id>nexus-all</id>
      <mirrorOf>*</mirrorOf>
      <url>https://nexus.internal/repository/maven-group/</url>
    </mirror>
  </mirrors>
</settings>
```

```
# npm — point to internal Artifactory
npm config set registry https://artifactory.internal/api/npm/npm-proxy/

# pip — point to internal PyPI proxy
pip install -r requirements.txt \
  --index-url https://artifactory.internal/api/pypi/pypi-proxy/simple/
```

```
// Jenkinsfile — use internal registry consistently
pipeline {
    agent any
    environment {
        MAVEN_OPTS = '-
Dmaven.repo.remote=https://nexus.internal/repository/maven-group/'
        NPM_CONFIG_REGISTRY = 'https://artifactory.internal/api/npm/npm-
proxy/'
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn --settings settings.xml clean package'
            }
        }
    }
}
```

---

### Case B: SNAPSHOT Dependency Resolving to Broken Build

```
# Symptom — different results on different runs
# Run 1: mylib-2.1.0-20241201.123456-1.jar  ← Works
# Run 2: mylib-2.1.0-20241202.093012-2.jar  ← Breaks
```

```
<!-- Problem: SNAPSHOT versions are mutable — they change daily -->
<dependency>
    <groupId>com.myorg</groupId>
    <artifactId>mylib</artifactId>
    <version>2.1.0-SNAPSHOT</version>  <!-- Non-deterministic in CI -->
</dependency>

<!-- Fix: Pin to a release version -->
<version>2.1.0</version>
```

```
// Jenkinsfile — disable snapshot updates for reproducibility
stage('Build') {
    steps {
        // -nsu = no snapshot updates from remote
        // Ensures same artifacts as last successful build
        sh 'mvn clean package -nsu --batch-mode'
    }
}
```

---

Case C: Transitive Dependency Conflict (Diamond Problem)

```
# Symptom at runtime (not compile time — sneaky!)
java.lang.NoSuchMethodError:
com.fasterxml.jackson.databind.ObjectMapper.readValue(...)
java.lang.ClassNotFoundException: org.apache.http.client.HttpClient

# Diagnosis — print full dependency tree
mvn dependency:tree -Dverbose | grep -E "omitted|conflict"
# or
gradle dependencies --configuration compileClasspath
```

```
Example conflict tree:
your-app
├── library-A:1.0 → requires jackson:2.12
└── library-B:1.0 → requires jackson:2.9   ← older version wins by
accident
                    jackson:2.9 (selected — wrong!)
```

```
<!-- Fix: Explicitly enforce the version you need -->
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>com.fasterxml.jackson.core</groupId>
```

```
            <artifactId>jackson-databind</artifactId>
            <version>2.15.2</version>
        </dependency>
    </dependencies>
</dependencyManagement>
```

```
// Gradle equivalent — force resolution
configurations.all {
    resolutionStrategy {
        force 'com.fasterxml.jackson.core:jackson-databind:2.15.2'
        failOnVersionConflict()  // Fail build if any conflict is
unresolved
    }
}
```

---

Case D: Corrupted Local Dependency Cache on Agent

```
# Symptom — random checksum errors
[ERROR] Checksum validation failed, expected <!DOCTYPE but was 9d7a...

# Diagnosis — look for partial/failed downloads
find ~/.m2 -name "*.lastUpdated" | head -20
find ~/.npm -name "*.lock" | head -20
```

```
// Fix 1: Delete stale cache markers before build
stage('Build') {
    steps {
        sh 'find ~/.m2 -name "*.lastUpdated" -delete'
        sh 'mvn clean package'
    }
}

// Fix 2: Use Docker volume for shared clean cache
pipeline {
    agent {
        docker {
            image 'maven:3.8.6-openjdk-17'
            args '-v jenkins-maven-cache:/root/.m2'
        }
    }
}

// Fix 3: Parameterized full cache wipe (not every build — expensive)
stage('Clean Cache') {
    when {
        expression { return params.CLEAN_CACHE == 'true' }
```

```
    }
    steps {
        sh 'rm -rf ~/.m2/repository'
        sh 'rm -rf ~/.npm/_cacache'
        sh 'rm -rf ~/.cache/pip'
    }
}
```

---

### Case E: Lockfile Missing or Not Committed

```
# Non-deterministic — resolves latest matching versions each time
npm install

# Deterministic — uses package-lock.json exactly ✅
npm ci

# npm ci fails fast if lockfile is missing or out of sync with
package.json
# This is what you WANT in CI — fail loudly, not silently
```

```
// Jenkinsfile — enforce lockfile usage
stage('Install Dependencies') {
    steps {
        sh 'npm ci'                                          // Node
        sh 'pip install -r requirements.txt --require-hashes' // Python
    }
}

// Add a validation stage to catch drift early
stage('Validate Lockfile') {
    steps {
        sh 'npm ci --dry-run && echo "Lockfile in sync"'
    }
}
```

---

### Prevention Checklist

```
✅ Always proxy external registries through internal Nexus/Artifactory
✅ Never use SNAPSHOT dependencies in production pipelines
✅ Pin all dependency versions — no ranges like ^1.0.0 or ~2.3
✅ Commit lockfiles (package-lock.json, Pipfile.lock, go.sum) to Git
✅ Use npm ci instead of npm install in CI
✅ Run dependency:tree in CI and fail on conflicts
✅ Cache dependencies in a named volume, not per-build ephemeral storage
✅ Set timeout on dependency download steps so hangs are caught early
```

---

### Interview Answer (Crisp)

> "Dependency failures fall into a few categories: registry availability, version mutability, conflict resolution, and cache corruption. The root fix for most of them is the same — proxy all external dependencies through an internal Nexus or Artifactory instance so you control availability, pin all versions including transitives using dependency management blocks, commit lockfiles, and use `npm ci` or `-nsu` Maven flags in CI to enforce determinism. For transitive conflicts, I run `mvn dependency:tree` to visualize the diamond problem and force the correct version via `dependencyManagement`."

---

### Where to Add in category11_cicd_jenkins_troubleshooting.md

Add this **after SCENARIO 11.25** and **before the Quick Reference table**.

Final order at the bottom of the file:

```
SCENARIO 11.25: Complete Pipeline Failure Analysis Framework
SCENARIO 11.26: Build Dependency Failures          ← INSERT HERE
Quick Reference: Most Common Jenkins Error Messages
Interview Cheat Sheet: What Interviewers Are Really Testing
```

Also add this row to the Quick Reference table:

```
| `Could not resolve dependency` | Registry down or version deleted |
Proxy via Nexus/Artifactory, pin all versions |
```

---

## Quick Reference: Most Common Jenkins Error Messages

| Error Message | Root Cause | Quick Fix |
|---|---|---|
| `No space left on device` | Agent disk full | Clean workspace, prune Docker images |
| `Permission denied` | Wrong user/group | Check jenkins user permissions |
| `Connection refused` | Service down or wrong port | Check connectivity, service status |
| `Could not transfer artifact` | Nexus/Artifactory auth issue | Rotate credentials |
| `Scripts not permitted to use` | Groovy sandbox restriction | Approve in Script Approval or use Pipeline steps |

| Error Message | Root Cause | Quick Fix |
|---|---|---|
| `Agent went offline` | Agent OOM/disk/network | Check agent resources |
| `timeout after X minutes` | Build hung waiting | Add `-y` flags, check for interactive prompts |
| `stash not found` | Stash expired or wrong name | Verify stash name, check controller disk |
| `Quality Gate FAILED` | Code quality below threshold | Fix issues or adjust thresholds |
| `Cannot connect to Docker daemon` | Socket permission | Add jenkins user to docker group |

## Interview Cheat Sheet: What Interviewers Are Really Testing

```
Junior level:  Can you read a stack trace and find the error?
Mid level:     Can you find root cause, not just symptom?
Senior level:  Can you prevent this class of problem entirely?
Staff level:   Can you design systems where these problems don't occur?
```

**Always end your troubleshooting answer with prevention:**

- "After fixing this, I would add [monitoring/alerting/automation] to catch it earlier next time"
- "I would add a runbook for this failure pattern"
- "I would write a post-mortem and share it with the team"

---

*Category 11 — Troubleshooting Scenarios | CI/CD & Jenkins Interview Mastery*